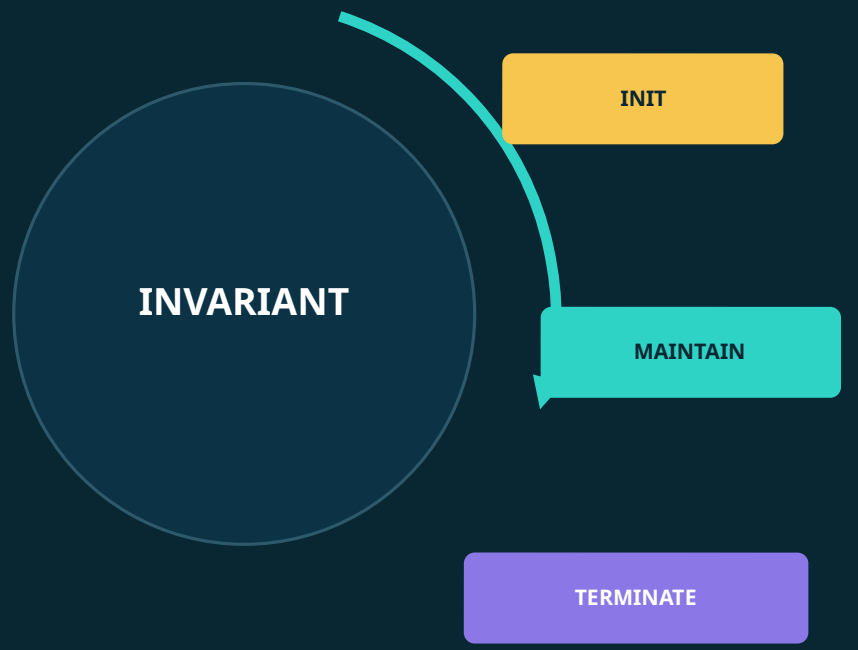
LECTURE 04

Loop Invariants and Iterative Correctness

How do we prove that a loop stays correct through every iteration?

Initialization • Maintenance • Termination



CDF OUTCOMES

What should you be able to do by the end?

Lecture 4 follows the CDF topic “Loop Invariants and Iterative Correctness.”

DEFINE

a loop invariant and explain what it says about a loop state

EXPLAIN

initialization, maintenance and termination

STRUCTURE

a correctness proof using the three invariant stages

APPLY

loop invariants to common iterative algorithms

EVALUATE

whether a proposed invariant/correctness claim is actually valid

PROVE

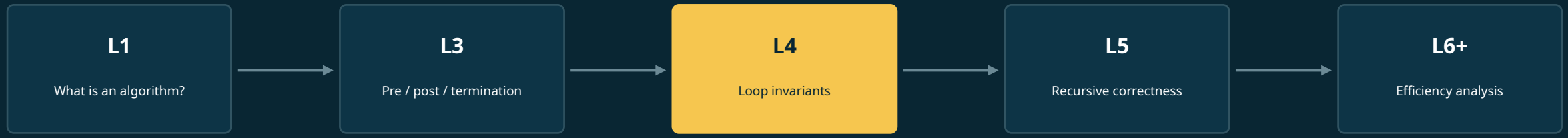
iterative summation and compare alternative iterative formulations

Correctness is not “the loop seems to work.” We need a statement that survives every iteration.

WHERE WE ARE

Where Lecture 4 fits

Lecture 3 gave us preconditions and postconditions. Lecture 4 explains how to bridge them across repeated execution.



Today we formalize the repeated part of an iterative algorithm.

MOTIVATION

Why are loops harder to prove correct?

A loop changes program state repeatedly. One proof must cover every possible number of iterations.

IF / ELSE — ONE DECISION

A small number of branches execute once. We can reason about each case separately.

LOOP — REPEATED CHANGES

The body may execute 0, 1, 10, or millions of times. Variable values keep changing.

THE PROOF PROBLEM

How can one argument cover every possible number of completed iterations?

We need a statement that stays true every time control reaches the same point in the loop.

That statement is a LOOP INVARIANT.

Important: the variables may change — the truth of the statement is what remains invariant.

SHORT EXAMPLE

A tiny loop to watch

Use n = 4. Track the program state at the START of every loop iteration.

SUM USING A WHILE LOOP

```
total ← 0
i ← 1

while i ≤ n
    total ← total + i
    i ← i + 1
```

PROGRAM STATE

The current values of the variables at a particular point during execution.

INITIAL STATE FOR n = 4

n = 4 i = 1 total = 0

Question: as i and total change, is there a relationship between them that keeps remaining true?

TRACE

Trace the loop for n = 4

Observe the state each time the while-condition is checked.

Checkpoint	i	total	What total represents	i ≤ 4?
Before iteration 1	1	0	sum before 1 = 0	Yes
Before iteration 2	2	1	1	Yes
Before iteration 3	3	3	1 + 2	Yes
Before iteration 4	4	6	1 + 2 + 3	Yes
After iteration 4	5	10	1 + 2 + 3 + 4	No — stop

i changes. total changes. But the relationship between them follows the same rule.

DISCOVER THE INVARIANT

What stays true at every loop check?

The values change, but the meaning of total follows one consistent pattern.

When i =	total =	Same statement written out
1	0	sum of integers 1 through 0
2	1	sum of integers 1 through 1
3	3	sum of integers 1 through 2
4	6	sum of integers 1 through 3
5	10	sum of integers 1 through 4

THE PATTERN

$$total = 1 + 2 + \cdots + (i - 1)$$

PLAIN ENGLISH

At the start of each iteration, total equals the sum of all integers already processed: 1 through i − 1.

Example: when i = 4, total = 6 because 1 + 2 + 3 = 6. Same truth; different values.

COMMON MISCONCEPTION

Invariant does NOT mean “nothing changes”

The variables can change on every iteration. The relationship that remains true is the invariant.

During the loop	Does it change?	Example
i	YES	1 → 2 → 3 → 4 → 5
total	YES	0 → 1 → 3 → 6 → 10
total = 1 + ⋯ + (i − 1)	NO — remains true	0=0, 1=1, 3=1+2, 6=1+2+3, …

INVARIANT = an unchanged TRUTH about a changing PROGRAM STATE

Not an invariant: “i = 1” or “total = 0” — those are only true at the beginning.

CORE IDEA

What is a loop invariant?

A statement about the program state that is true at a specific point in every iteration of the loop.

LOOP INVARIANT A property that is established before the first iteration, preserved by every iteration, and useful when the loop ends.

Think: “No matter how many iterations have already happened, this statement is still true.”

CONNECTION

Precondition, invariant and postcondition

They play different roles in one correctness argument.

PRECONDITION P

What must be true before the algorithm starts?

INVARIANT I

What remains true at the chosen loop point before/after every iteration?

POSTCONDITION Q

What must be true when the algorithm finishes?

$$P \rightarrow I \rightarrow \ldots \text{repeated} \ldots \rightarrow I \wedge \neg \text{guard} \rightarrow Q$$

The invariant is the bridge that carries a correctness fact through the loop.

PROOF STRUCTURE

The three stages of a loop-invariant proof

Use the same structure every time.

1 INITIALIZATION

Show that the invariant is true before the first iteration.

Question: did we establish I correctly?

2 MAINTENANCE

Assume I is true before an iteration. Show that after executing the loop body, I is still true for the next iteration.

3 TERMINATION

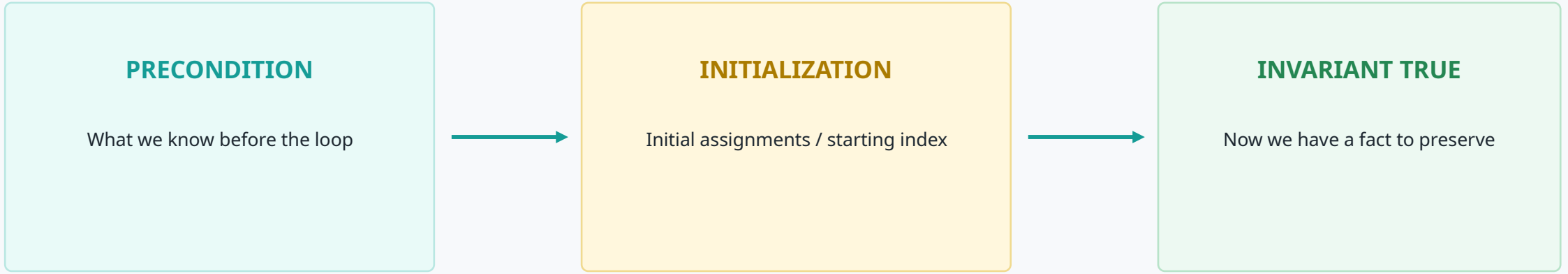
When the loop stops, combine the invariant with the false loop condition to derive the postcondition.

Initialization + Maintenance establishes partial correctness. Termination of the loop is still required for total correctness.

STAGE 1

Initialization: make the invariant true once

Before preservation matters, the invariant must hold at the very beginning.



Initialization is usually the easiest stage—but if it fails, the entire proof collapses.

STAGE 2

Maintenance: assume it, execute once, recover it

This is the inductive heart of the proof.

Assume I is true before iteration k. → **Execute the loop body once.**

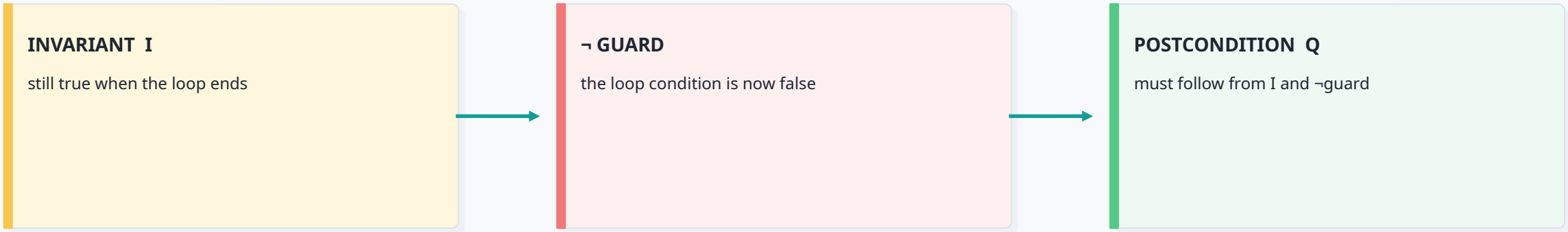
SHOW: if I was true before this iteration, then I is true again for the next iteration.

We are not proving the whole loop at once. We prove that one iteration cannot destroy the invariant.

STAGE 3

Termination: use the invariant when the guard becomes false

The invariant alone is not the final answer; it must help derive the postcondition.



$$I \wedge \neg guard \Rightarrow Q$$

Separate question: do we know the loop actually reaches this point?

TOTAL CORRECTNESS

A loop invariant proves preservation — not termination by itself

For total correctness, we also need an argument that the loop cannot continue forever.

INVARIANT

supports the “right answer” argument

TERMINATION MEASURE

a finite quantity moves toward the stopping condition

TOTAL CORRECTNESS

correct result + guaranteed finish

Example: in a loop with $i = 1,2,\ldots,n$, the quantity $n - i$ decreases until the guard fails.

This “decreasing measure” is often called a loop variant or termination measure.

MAIN CDF EXAMPLE

Worked Example 1: Iterative Summation

Prove that SUM(n) returns 1 + 2 + … + n.

```
SUM(n)

total ← 0
for i ← 1 to n
    total ← total + i
return total
```

SPECIFICATION

Precondition
n is a positive integer

Postcondition
total = 1 + 2 + … + n

SUM(n)

Step 1: choose a useful invariant

The invariant should describe exactly what total means at the start of each iteration.

At the start of iteration i: total = 1 + 2 + … + (i − 1)

English: before adding i, total already contains the sum of all integers that came before i.

Equivalent compact notation: total = Σ(k=1 to i−1) k

A good invariant gives a precise meaning to the variables at the chosen loop point.

READ THE MATH

What does the summation invariant mean?

Translate the mathematics into English before proving anything.

Expression	Read it as	Meaning
i	current loop index	the next integer that will be added
i − 1	one less than i	the last integer already included in total
Σ	sum	add a sequence of values
Σ(k=1 to i−1) k	sum k from 1 to i−1	1 + 2 + … + (i−1)

Always pair the formal invariant with an English interpretation.

SUM(n) — INITIALIZATION

Stage 1: Initialization

Is the invariant true before the first iteration?

STARTING STATE

total ← 0
i ← 1

→

INVARIANT REQUIRES

total = 1 + … + (i−1)

→

SUBSTITUTE i = 1

total = sum of no previous values = 0

Before iteration 1: total = 0 ✓

English: before adding the first number, the sum of the numbers already processed is zero.

SUM(n) — MAINTENANCE

Stage 2: Maintenance

Assume the invariant is true before iteration i; show it is true for iteration i+1.

ASSUME BEFORE

total = 1 + … + (i−1)

the invariant is true

EXECUTE BODY

```
total ← total + i
```

Now total = 1 + … + i

NEXT ITERATION

i becomes i + 1

Invariant now says total = 1 + … + ((i+1)−1) = 1 + … + i ✓

One execution of the loop body preserves the invariant.

SUM(n) — TERMINATION

Stage 3: Termination

When the loop stops, use the invariant to derive the required sum.

LOOP STOPS

After n has been processed, the next index is i = n + 1.

INVARIANT STILL TRUE

total = 1 + 2 + … + (i − 1)

SUBSTITUTE

i = n + 1
⇒ total = 1 + 2 + … + n

Therefore the postcondition holds: total = Σ(k=1 to n) k

Termination is guaranteed because i increases by 1 and can pass n after finitely many iterations.

PROOF COMPLETE

SUM(n) is totally correct

For every positive integer n, the algorithm returns 1 + 2 + … + n and terminates.

INITIALIZATION

Invariant true before iteration 1

MAINTENANCE

Every iteration preserves the invariant

TERMINATION

Invariant + stopped guard ⇒ postcondition

This three-stage structure is reusable for many iterative algorithms.

MAX(A): start with one concrete array

Before proving the loop, watch what largest means as the processed prefix grows.

ALGORITHM

```
MAX(A)

largest ← A[0]
for i ← 1 to n−1
    if A[i] > largest
        largest ← A[i]
return largest
```

CONCRETE INPUT

A = [8, 3, 12, 5, 10]

INITIAL STATE

largest = 8 i = 1

Only A[0] has been processed so far.

Question: what should largest represent at every later loop check?

TRACE

Trace MAX(A) on A = [8, 3, 12, 5, 10]

Observe largest at the start of each iteration and how the processed prefix grows.

Checkpoint	i	A[i]	largest before	largest after	Processed prefix
Before iter. 1	1	3	8	8	[8,3]
Before iter. 2	2	12	8	12	[8,3,12]
Before iter. 3	3	5	12	12	[8,3,12,5]
Before iter. 4	4	10	12	12	[8,3,12,5,10]
After loop	5	—	12	—	entire array

largest changes only when the new element is bigger than everything already seen.

What stays true in MAX(A)?

Focus on the already processed prefix A[0..i−1].

PATTERN IN THE TRACE

i = 1 → processed [8] largest = 8
i = 2 → processed [8,3] largest = 8
i = 3 → processed [8,3,12] largest = 12
i = 4 → processed [8,3,12,5] largest = 12

THE INVARIANT

largest = max(A[0 .. i−1])

PLAIN ENGLISH

At the start of iteration i, largest is the largest value among all array elements already processed.

Example: when i = 4, largest = 12 because max([8,3,12,5]) = 12.

WORKED EXAMPLE 2

Revisit MAX(A) from Lecture 3

Now we turn the informal correctness reasoning into a loop-invariant argument.

```
MAX(A)

largest ← A[0]
for i ← 1 to n−1
    if A[i] > largest
        largest ← A[i]
return largest
```

PRECONDITION

A is finite, non-empty, and its elements are comparable.

POSTCONDITION

$m \in A$ and $\forall x \in A, m \geq x$

PROOF QUESTION

What statement about largest is true before every iteration and becomes exactly the desired maximum when the loop ends?

MAX(A)

Choose the MAX invariant

At the start of iteration i, the algorithm has already processed A[0] through A[i−1].

$$\text{largest} = \max(A[0 \,..\, i-1])$$

English: largest is the maximum value among all array elements examined so far.

Notice how this is stronger and more useful than merely saying “largest is large.”

MAX(A) — INITIALIZATION

Initialization for MAX(A)

Before iteration i = 1, exactly one array element has been processed.

INITIAL ASSIGNMENT

largest ← A[0]

PROCESSED PREFIX

A[0 .. i−1] = A[0 .. 0]

INVARIANT

largest = max(A[0]) = A[0] ✓

The first element is trivially the maximum of the one-element prefix containing only itself.

MAX(A) — MAINTENANCE

Maintenance for MAX(A)

Assume largest is the maximum of A[0..i−1]. Now inspect A[i].

CASE 1

A[i] ≤ largest

Do not update. largest remains the maximum of the enlarged prefix.

CASE 2

A[i] > largest

Set largest ← A[i]. The new element becomes the maximum of the enlarged prefix.

AFTER EITHER CASE

largest = max(A[0..i])

Exactly the invariant required for the next iteration.

Every iteration expands the processed prefix by one element while preserving the meaning of largest.

Termination for MAX(A)

When the loop finishes, the processed prefix is the entire array.

At loop exit: $i = n$

Invariant: $largest = \max(A[0 .. i-1])$



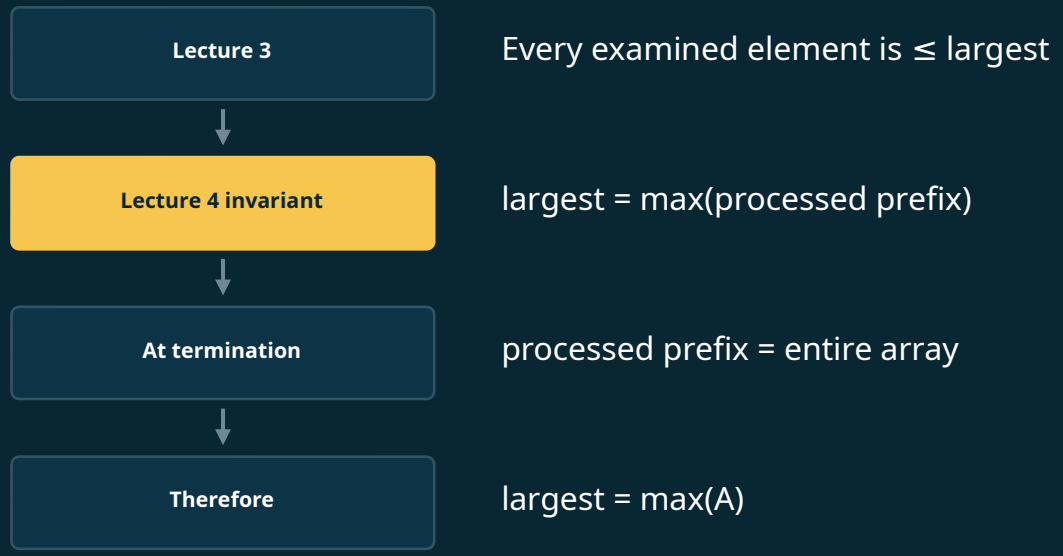
$$largest = \max(A[0 .. n-1]) = \max(A)$$

The postcondition follows directly. Since i advances toward n, the loop also terminates.

CONNECTION

Lecture 3 reasoning → Lecture 4 proof

The informal idea “largest is the biggest seen so far” becomes a formal invariant.



WORKED EXAMPLE 3

Linear Search: start with a concrete search

This invariant records which part of the array has already been ruled out.

ALGORITHM

```
LINEAR-SEARCH(A, x)

for i ← 0 to n−1
    if A[i] = x
        return i
return NOT-FOUND
```

CONCRETE INPUT

A = [27, 14, 81, 3, 45, 62] x = 45

WHAT CHANGES?

i moves right through the array.
The prefix before i contains values already checked.

Question: what can we truthfully say about x in the prefix A[0..i−1]?

TRACE

Trace Linear Search for x = 45

At each new loop check, everything before i has already been compared with x.

Checkpoint	i	A[i]	Already checked prefix	What we know
Before iter. 0	0	27	empty	nothing ruled out
Before iter. 1	1	14	[27]	45 not in [27]
Before iter. 2	2	81	[27,14]	45 not in first 2
Before iter. 3	3	3	[27,14,81]	45 not in first 3
Before iter. 4	4	45	[27,14,81,3]	45 not in first 4
At iter. 4	4	45	—	match → return 4

The search succeeds at index 4; every earlier position has already been ruled out.

What stays true in Linear Search?

The invariant describes absence from the processed prefix.

PATTERN

i = 1 → x not in A[0]
i = 2 → x not in A[0..1]
i = 3 → x not in A[0..2]
i = 4 → x not in A[0..3]

THE INVARIANT

$$x \notin A[0 \,..\, i-1]$$

PLAIN ENGLISH

At the start of iteration i, x has not been found in any position already examined.

If i reaches n without a return, the ruled-out prefix is the entire array.

WORKED EXAMPLE 3

Linear Search: a different kind of invariant

The invariant can describe what has NOT been found yet.

```
LINEAR-SEARCH(A, x)

for i ← 0 to n−1
    if A[i] = x
        return i
return NOT-FOUND
```

PRECONDITION

A is finite; equality comparison with x is defined.

POSTCONDITION

Return an index i with A[i]=x, or NOT-FOUND only if x is absent from A.

CANDIDATE INVARIANT

At the start of iteration i, x does not occur in A[0 .. i−1].

LINEAR SEARCH

Reason through the Linear Search invariant

The invariant grows the “already ruled out” prefix.

INITIALIZATION

At i = 0, no elements have been examined. The empty prefix contains no x. ✓

MAINTENANCE

If A[i] ≠ x, then after checking it, x is absent from A[0..i]. The invariant holds for i+1.

TERMINATION

If the loop reaches i = n without returning, x is absent from A[0..n−1] — the entire array.

If A[i] = x at any step, returning i immediately satisfies the “found” part of the postcondition.

COMMON MISTAKES

Not every true-looking statement is a useful invariant

A proposed invariant must be true initially, preserved, and strong enough to prove the postcondition.

TOO WEAK

“total ≥ 0”

May be true for positive summation, but it does not tell us which sum total represents.

FALSE INITIALLY

“total = 1 + … + i” before iteration i

At i=1, total is 0, not 1.

NOT PRESERVED

A statement may start true but become false after the loop body. Then it is not invariant.

A useful invariant must pass all three tests: initialization, maintenance, and usefulness at termination.

EVALUATE

Can you judge a proposed invariant?

Use the proof structure instead of intuition.

```
for i ← 1 to n
    total ← total + i
```

Candidate: “At the start of iteration i, total = 1 + … + i.”

TEST 1

Initialization: for i=1, total=0. Candidate requires total=1. FAIL.

TEST 2

Because it already fails initialization, it cannot be the invariant for this chosen loop point.

FIX

Use total = 1 + … + (i−1) before iteration i.

TECHNIQUE SELECTION

Different iterative formulations can be equally correct

Optimization or reformulation is acceptable only if correctness is preserved.

```
SUM-UP(n)
total ← 0
for i ← 1 to n
    total ← total + i
return total
```

```
SUM-DOWN(n)
total ← 0
for i ← n downto 1
    total ← total + i
return total
```

```
SUM-PAIR(n)
total ← 0
process two terms per step
handle final odd term
return total
```

All three aim for the same postcondition, but their invariants are naturally written differently.

Never assume a “faster-looking” loop is correct just because it resembles a correct one. Re-establish the invariant.

ALTERNATIVE EXAMPLE

SUM-DOWN(n): same answer, opposite direction

Trace first. The direction changes what 'already processed' means.

ALGORITHM

```
SUM-DOWN(n)

total ← 0
i ← n
while i ≥ 1
    total ← total + i
    i ← i − 1
return total
```

CONCRETE INPUT

n = 4

INITIAL STATE

i = 4 **total = 0**

No values have been added yet.

Because we move downward, the processed values will be the numbers larger than i.

Trace SUM-DOWN for n = 4

Watch how total accumulates the upper part of the range.

Checkpoint	i	total	Numbers already included
Before iter. 1	4	0	none
Before iter. 2	3	4	4
Before iter. 3	2	7	3 + 4
Before iter. 4	1	9	2 + 3 + 4
After loop	0	10	1 + 2 + 3 + 4

The meaning of total now refers to the upper end of the range.

ALTERNATIVE PROOF

Example: SUM-DOWN(n) needs a different invariant

Same result. Different direction. Different description of what has already been processed.

At the start of iteration i: **total = (i+1) + (i+2) + … + n**

English: before adding i, total already contains every number larger than i up to n.

When the loop finishes after adding 1, the processed set is again {1,2,…,n}; the postcondition is unchanged.

SUM-DOWN: initialization, maintenance, termination

Once the invariant is visible, use the same three-stage proof structure.

INITIALIZATION

At i = n, total = 0. The sum from i+1 to n is empty, so the invariant holds.

MAINTENANCE

Assume total = (i+1)+…+n. Add i, then decrement i. The same pattern holds for the new i.

TERMINATION

The loop stops with i = 0. The invariant becomes total = 1+2+…+n.

The direction changed, so the invariant changed—but the proof pattern did not.

KEY PERSPECTIVE

Correctness must survive optimization

Changing the loop structure changes what must be proved—even when the final goal is the same.

1. State the same postcondition
2. Identify the new loop state
3. Write an invariant for the new formulation
4. Re-prove initialization / maintenance / termination
5. Only then compare efficiency

WORKED EXAMPLE 4

PRODUCT: compute 1 × 2 × … × n

Use n = 4. First trace the loop before trying to write an invariant.

ALGORITHM

```
PRODUCT(n)

product ← 1
i ← 1
while i ≤ n
    product ← product × i
    i ← i + 1
return product
```

CONCRETE INPUT

n = 4

INITIAL STATE

i = 1 **product = 1**

Question: after some factors have been processed, what exactly does product contain?

Trace PRODUCT for n = 4

At each checkpoint, product contains every factor already processed.

Checkpoint	i	product	Factors already included
Before iter. 1	1	1	none (empty product = 1)
Before iter. 2	2	1	1
Before iter. 3	3	2	1 × 2
Before iter. 4	4	6	1 × 2 × 3
After loop	5	24	1 × 2 × 3 × 4

The value changes, but its meaning follows one consistent rule.

What stays true in PRODUCT?

Describe the factors already processed at the start of iteration i.

PATTERN

i = 2 → product = 1
i = 3 → product = 1 × 2
i = 4 → product = 1 × 2 × 3

THE INVARIANT

product = 1 × 2 × … × (i−1)

PLAIN ENGLISH

At the start of iteration i, product is the product of all integers already processed: 1 through i−1.

The empty product is defined as 1, which makes initialization at i = 1 work.

PRODUCT: prove the invariant in three stages

The structure is identical to SUM; only the accumulated operation changes.

INITIALIZATION

At i = 1, product = 1. This is the product of the empty set of previously processed factors.

MAINTENANCE

Assume product = 1×…×(i−1). Multiply by i, then increment i. The new product equals 1×…×i.

TERMINATION

The loop stops with i = n+1. The invariant gives product = 1×2×…×n.

A decreasing measure such as n − i + 1 also shows that the loop terminates.

COUNT POSITIVES: count positive values in an array

Trace a concrete array and watch what count means after each processed element.

ALGORITHM

```
COUNT-POSITIVE(A)

count ← 0
for i ← 0 to n−1
    if A[i] > 0
        count ← count + 1
return count
```

CONCRETE INPUT

A = [3, −1, 0, 7, −4, 2]

INITIAL STATE

i = 0 count = 0

Question: what does count represent in the prefix A[0..i−1]?

Trace COUNT POSITIVES

The prefix before i has been completely processed at each loop check.

Checkpoint	i	A[i]	count before	count after	Positives seen
Before 0	0	3	0	1	3
Before 1	1	−1	1	1	3
Before 2	2	0	1	1	3
Before 3	3	7	1	2	3, 7
Before 4	4	−4	2	2	3, 7
Before 5	5	2	2	3	3, 7, 2

After the final element, count = 3.

What stays true in COUNT POSITIVES?

The accumulator count describes a property of the processed prefix.

THE PATTERN

Before i = 1 → one positive in A[0..0]
Before i = 3 → one positive in A[0..2]
Before i = 5 → two positives in A[0..4]

THE INVARIANT

count = # positives in A[0 .. i−1]

PLAIN ENGLISH

At the start of iteration i, count equals the number of positive values among all elements already processed.

The symbol # here simply means 'number of'.

PROOF SUMMARY

COUNT POSITIVES: prove the prefix-count invariant

Check initialization, one-step preservation, and the final processed prefix.

INITIALIZATION

At i = 0, the processed prefix is empty and count = 0. There are zero positive values.

MAINTENANCE

If A[i] > 0, increment count; otherwise leave it unchanged. Either way count becomes the positives in A[0..i].

TERMINATION

When i = n, the processed prefix is the entire array, so count is the total number of positive values.

The invariant talks about the processed prefix—not about fixed variable values.

MIN(A): adapt the MAX reasoning

The structure is nearly identical to MAX, but the comparison direction changes.

ALGORITHM

```
MIN(A)

smallest ← A[0]
for i ← 1 to n−1
    if A[i] < smallest
        smallest ← A[i]
return smallest
```

CONCRETE INPUT

A = [8, 3, 12, 5, 1, 10]

INITIAL STATE

smallest = 8 i = 1

Question: what should smallest mean after each larger processed prefix?

Trace MIN(A)

Update smallest only when the newly processed value is smaller.

Checkpoint	i	A[i]	smallest before	smallest after
Before 1	1	3	8	3
Before 2	2	12	3	3
Before 3	3	5	3	3
Before 4	4	1	3	1
Before 5	5	10	1	1
After loop	6	—	1	—

By the end, smallest = 1, the minimum of the entire array.

What stays true in MIN(A)?

As with MAX, focus on the minimum of the already processed prefix.

PATTERN

i = 2 → min([8,3]) = 3
i = 3 → min([8,3,12]) = 3
i = 5 → min([8,3,12,5,1]) = 1

THE INVARIANT

smallest = min(A[0 .. i−1])

PLAIN ENGLISH

At the start of iteration i, smallest is the smallest value among all array elements already processed.

This is the MAX invariant with 'maximum' replaced by 'minimum'.

PROOF SUMMARY

MIN(A): prove the prefix-minimum invariant

The proof mirrors MAX(A), but uses the < comparison.

INITIALIZATION

Before i = 1, only A[0] has been processed, and smallest = A[0].

MAINTENANCE

Compare A[i] with smallest. Updating only when A[i] is smaller makes smallest the minimum of A[0..i].

TERMINATION

When i = n, A[0..n−1] is the entire array, so smallest = min(A).

Precondition reminder: A must be finite, non-empty, and its elements comparable.

CHECKLIST

A systematic checklist for iterative correctness

Use this whenever you are asked to prove a loop correct.

1. **SPECIFY** — Write the precondition and postcondition.
2. **CHOOSE I** — State the invariant at an exact loop point.
3. **INITIALIZE** — Prove I before the first iteration.
4. **MAINTAIN** — Assume I; execute one iteration; recover I.
5. **TERMINATE** — Combine I with the false guard to derive Q.
6. **FINISH** — Give a termination argument for total correctness.

EVALUATION

Tracing helps you discover an invariant — it does not prove it

Use examples to see a pattern, then prove the pattern survives every iteration.

TRACE

SUM(5) gives 15.
MAX([8,3,12]) gives 12.

Useful evidence and intuition.

INVARIANT

States what is true for an arbitrary iteration i, not only the iterations we happened to test.

PROOF

Initialization + maintenance + termination covers all valid executions represented by the specification.

A trace can suggest the invariant. The invariant proof establishes correctness.

Five ideas to carry forward

Loop invariants turn repeated execution into a structured correctness proof.

01	**Invariant**	a fact that remains true at the chosen loop point
02	**Initialization**	establish it before the first iteration
03	**Maintenance**	prove one iteration preserves it
04	**Termination**	use invariant + false guard to reach the postcondition
05	**Total correctness**	correct result + guaranteed termination

NEXT
Recursive Correctness